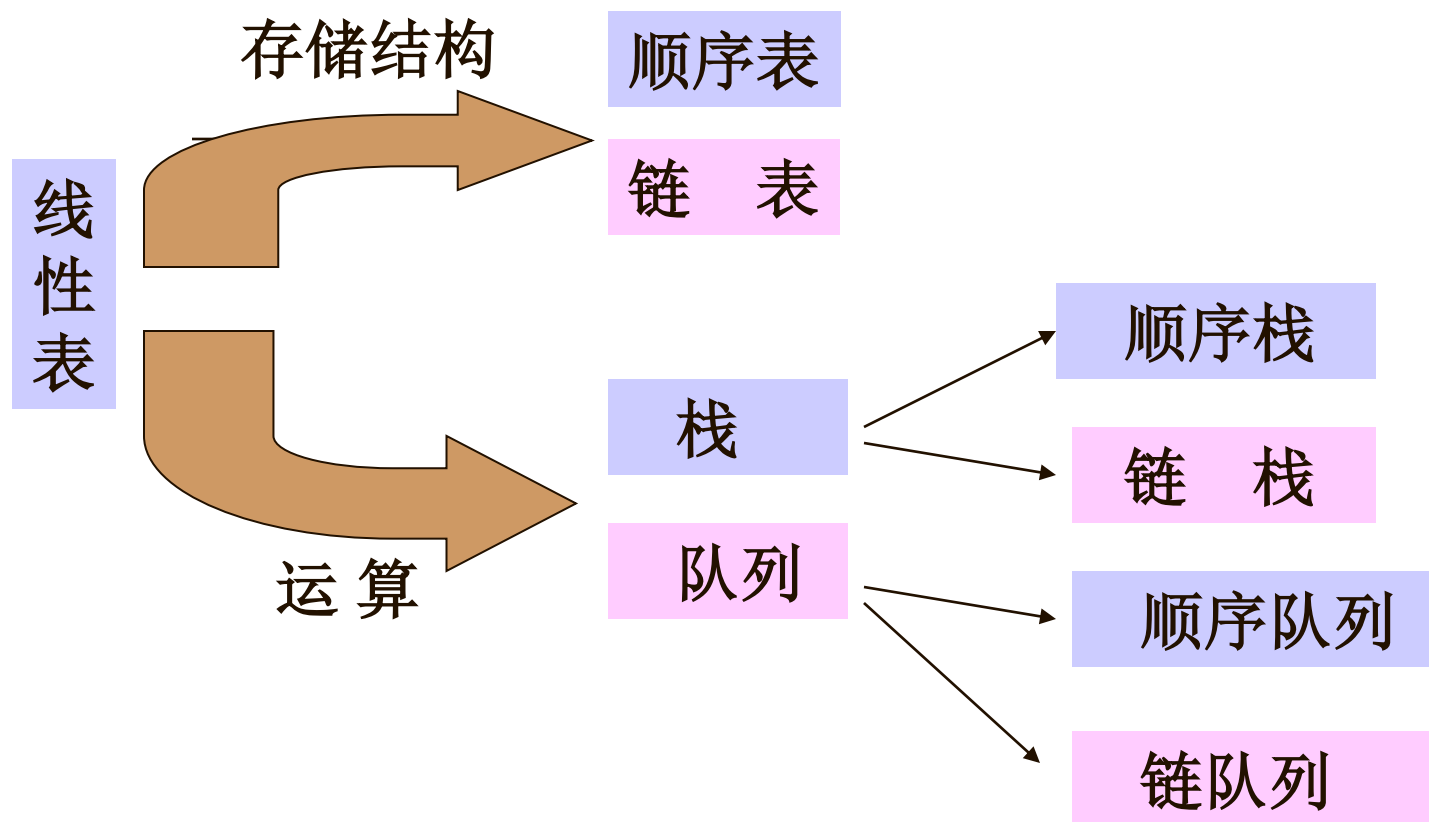


回顾



数据结构与算法设计

第四章 串

- 4.1 [串类型的定义](#)
- 4.2 [串的实现和表示](#)
- 4.3 [串的模式匹配算法](#)

第四章 串

上海自来水来自海上

两相思
宋 李禺

枯眼望遥山隔水，
往来曾见几心知？
壶空怕酌一杯酒，
笔下难成和韵诗。
途路阻人离别久，
讯音无雁寄回迟。
孤灯夜守长寥寂，
夫忆妻兮父忆儿。



儿忆父兮妻忆夫，
寂寥长守夜灯孤。
迟回寄雁无音讯，
久别离人阻路途。
诗韵和成难下笔，
酒杯一酌怕空壶。
知心几见曾来往，
水隔山遥望眼枯。

第四章 串

lover

over

friend

end

believe

lie

4.1 串类型的定义

串 (string) 即字符串，是由零个或多个字符组成的有限序列，是数据元素为单个字符的特殊线性表。

记为： $s = 'a_1 a_2 \cdots a_n'$ ($n \geq 0$)


串名 串值（用 ‘ ’ 括起来）

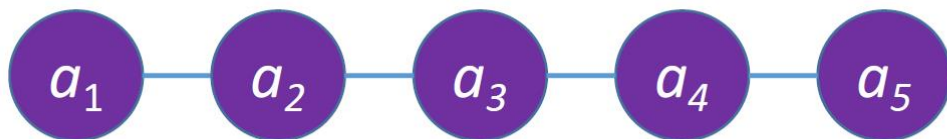
隐含结束符 ‘\0’ ，
即ASCII码NULL

其中：

- s 是串名字
- $'c_1 c_2 c_3 \cdots c_n'$ 是串值
- c_i 是串中字符
- n 是串的长度，表示串中字符的数目

4.1 串类型的定义

串是一种特殊的线性表，数据元素之间呈线性关系



串的数据对象限定为字符集（如中文字符、英文字符、数字字符、标点字符等）

串的基本操作，如增删改查等通常以子串为操作对象



我们不一样

4.1 串类型的定义

串的表示：用一对单引号括起来

例：S="HelloWorld! “

T="HUAWEI Mate 70 Pro?"

有的地方用双引号(如Java、C)
有的地方用单引号(如Python)

空串：零个字符的串称为**空串**记作“ $\emptyset$ ”

空白串：由一个或多个**空格符**组成的串

问：空串和空白串有无区别？

答：有区别。

空串(Null String)是指长度为零的串；

空白串(Blank String)，是指包含一个或多个空白字符 ‘ ’ (空格键)的字符串。

4.1 串类型的定义

子串： 串中任意个连续的字符组成的子序列

主串： 包含子串的串

字符在串中的位置： 字符在序列中的序号

子串在串中的位置： 子串的第一个字符在主串中的位置

串的长度： 串中字符的个数

一个串中任意个连续字符组成的子序列(含空串)称为该串的**子串**。例如，“a”、“ab”、“abc”和“abcd”等都是“abcde”的子串。

4.1 串类型的定义

例1：现有以下4个字符串：

$a = \text{'BEI'}$ $b = \text{'JING'}$ $c = \text{'BEIJING'}$ $d = \text{'BEI JING'}$

问：① 它们各自的长度？ $a = 3$, $b = 4$, $c = 7$, $d = 8$

② a 是哪个串的子串？在主串中的位置是多少？

a 是 c 和 d 的子串，在 c 和 d 中的位置都是1

③ 空串是哪个串的子串？ a 是不是自己的子串？

“空串是任意串的子串；任意串 S 都是 S 本身的子串，除 S 本身外， S 的其他子串称为 S 的真子串。”

4.1 串类型的定义

串相等：当且仅当两个串的长度相等并且各个对应位置上的字符都相同时，这两个串才是相等的

串的操作：以“串的整体”为操作对象

①串的元素全为字符

②线性表的操作基本是单个元素；串的基本操作是“串的整体”

4.1 串类型的定义

ADT String{

Objects: $D=\{a_i \mid a_i \in \text{CharacterSet}, i=1, 2, \dots, n, n \geq 0\}$

Relations: $R1=\{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2, \dots, n \}$

functions:

//至少有13种基本操作

基本
操作
子集

StrAssign(&T, chars) // 串赋值, 生成值为chars的串T

StrCompare(S,T) // 串比较, 若 $S > T$, 返回值大于0...

StrLength(S) // 求串长, 即返回串S中的元素个数

Concat(&T, S1, S2) // 串连接, 用T返回 $S1+S2$ 的新串

SubString(&Sub, S, pos, len) // 求S中pos起长度为len的子串

.....

Index(S, T, pos) //子串定位函数 (模式匹配), 返回位置

Replace(&S, T,V) // 用子串V替换子串T

}ADT String

C语言中已有类似串运算函数!

4.1 串类型的定义

注：用C处理字符串时，要调用标准库函数 `#include<string.h>`

C语言

串比较： `int strcmp(char *s1,char *s2);`

求串长： `int strlen(char *s);`

串连接： `char strcat(char *to,char *from)`

子串T定位： `char strchr(char *s,char *c);`

.....

类C

`StrCompare(S,T)`

`StrLength(S)`

`Concat(&T, S1, S2)`

`Index(S, T, pos)`

4.1 串类型的定义

例1：设 $s = \text{'I AM A STUDENT'}$, $t = \text{'GOOD'}$, $q = \text{'WORKER'}$ 。求：

$\text{StrLength}(s) = 14$ // 参见P71
 $\text{StrLength}(t) = 4$
 $\text{SubString}(\&\text{sub}, s, 8, 7) = \text{'STUDENT'}$
 $\text{SubString}(\&\text{sub}, t, 2, 1) = \text{'O'}$
 $\text{Index}(s, \text{'A'}) = 3$
 $\text{Index}(s, t) = 0$
 $\text{Replace}(\&s, \text{'STUDENT'}, q) = \text{'I AM A WORKER'}$
(s 中没有 $t = \text{'GOOD'}$!)

$\text{Index}(S, T, \text{pos})$
// 返回子串T在pos
之后的位置

$\text{Replace}(\&S, T, V)$
// 用子串V替换子串T

4.1 串类型的定义

提问：当s='I(A)M(A)STUDENT'时，

INDEX (s,'A', pos) =3，若想搜索后面那个 'A'怎么办？

答：根据教材P71倒1行的函数说明，INDEX (s,'A') 返回的只是“第一次”出现的位置。

如果还要搜索后面的A，则pos变量要跟着变才行。

也就是说，要把得到的“第一次”位置再代入INDEX (s,'A', pos+1) 函数中循环操作才行。

4.1 串类型的定义

例2: 设 $s = \text{'I AM A STUDENT'}$, $t = \text{'GOOD'}$, 求:
 $\text{Concat}(\text{SubString}(s, 6, 2), \text{Concat}(t, \text{SubString}(s, 7, 8))) = ?$

解: 因为

$\text{SubString}(s, 6, 2) = \text{'A'}$;

$\text{SubString}(s, 7, 8) = \text{'STUDENT'}$

$\text{Concat}(t, \text{SubString}(s, 7, 8)) = \text{'GOOD STUDENT'}$

所以:

$\text{Concat}(\text{SubString}(s, 6, 2), \text{Concat}(t, \text{SubString}(s, 7, 8)))$

$= \text{'A GOOD STUDENT'}$

4.2 串的实现

- 1. 定长顺序存储表示
- 2. 堆分配存储表示
- 3. 串的块链存储表示
- 4. 串的基本操作

4.2 串的实现

串是一种特殊的线性表，在非紧缩格式中，它的每个结点仅由一个字符组成，因此存储串的方法也就是存储线性表的一般方法。存储串最常用的方式是采用顺序存储，即把串的字符顺序地存储在内存一片相邻的空间，这称为**顺序串**。

强调：串与线性表的运算有所不同，是以“**串的整体**”作为操作对象，例如查找某子串，在主串某位置上插入一个子串等。

4.2 串的实现

串的物理存储表示方法：

顺序存储

- 定长顺序存储表示

——用一组地址连续的存储单元存储串值的字符序列，属静态存储方式。

- 堆分配存储表示

——用一组地址连续的存储单元存储串值的字符序列,但存储空间是在程序执行过程中动态分配而得。

4.2 串的实现

串的物理存储表示方法：

链
式
存
储

- 串的块链存储表示

——链式方式存储

1. 定长顺序存储表示

- 静态分配

- 每个串预先分配一个固定长度的存储区域。
- 实际串长可在所分配的固定长度区域内变动
 - 以下标为0的数组分量存放串的实际长度——PASCAL
 - 在串值后加入” \0” 表示结束，此时串长为隐含值

- 用定长数组描述：

```
#define MAXSTRLEN 255 //最大串长
```

```
typedef unsigned char SString[MAXSTRLEN + 1]
```

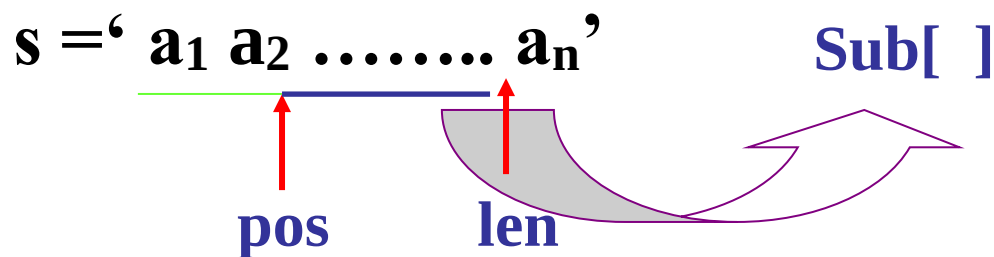
```
//0号单元存放串的长度
```

1. 定长顺序存储表示

例：用顺序存储方式编写求子串函数

SubString(&Sub,S,pos,len)

将串S中从第pos个字符开始、长度为len的字符序列复制到串Sub中。（注：考虑到函数的通用性，应当让串Sub的预留长度与S一样）



1. 定长顺序存储表示

例：用顺序存储方式编写求子串函数SubString(&Sub,S,pos,len)

```
Status SubString(SString &sub, SString S, int pos, int len )  
{ if(pos<1 || pos>S[0] || len<0 || len>S[0]-pos+1) return ERROR;  
  //若pos和len参数越界，则告警  
  Sub[1.....len]=S[pos.....pos+len-1];  
  Sub[0]=len; return OK;  
}
```

子串长度

讨论：想存放超长字符串怎么办？
改用动态分配的一维数组——堆

2. 堆分配存储表示

- 以一组地址连续的存储单元存放串值字符序列；
- 存储空间**动态分配**，用malloc()和free()来管理
- P75示例

思路： 利用malloc函数合理预设串长空间。

特点： 若在操作中串值改变，还可以利用**realloc**函数按新串长度增加空间（即**动态数组概念**）。

2. 堆分配存储表示

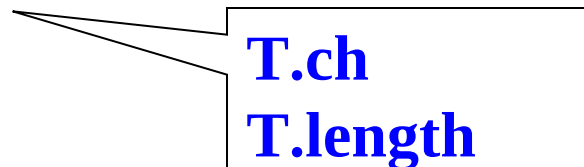
堆T的存储结构描述

```
typedef struct {
```

```
    char *ch; //若非空串,按串长分配空间; 否则ch=NULL
```

```
    int length; //串长度
```

```
}HString
```



4. 串的基本操作

- 求串长 `int StrLength(HString S)`
- 串比较 `int StrCompare(HString S, HString T)`
- 串联接 `Status Concat(HString &S, HString S1, HString S2)`
- 串清空 `Status ClearString(HString &S)`
- 求子串 `Status SubString(HString &Sub, HString S, int pos, int len)`
- 串插入 `Status StrInsert(HString &S, int pos, HString T)`
- 串赋值 `Status StrAssign(HString &S, char *chars)`
- 串定位
- 删除
- 置换

4. 串的基本操作

```
int StrLength(HString S)
```

```
//求串的长度
```

```
{
```

```
    return S.length;
```

```
}
```

4. 串的基本操作

```
int StrCompare(HString S, HString T)
//比较两个串，若相等返回0
{
    int i;
    for (i=0; i<S.length && i<T.length; ++i)
        if (S.ch[i] != T.ch[i]) return S.ch[i]-T.ch[i];
    return S.length-T.length;
}
```

4. 串的基本操作

Status Concat (HString &S, HString S1, HString S2)

//用S返回由S1和S2联接而成的新串

```
{ int j;
  if (!(S.ch = (char*)malloc((S1.length+S2.length)*sizeof(char))))
    exit(OVERFLOW);
  for (j=0; j<=S1.length-1; j++)
    { S.ch[j]=S1.ch[j]; }
  S.length=S1.length+S2.length;
  for (j=0; j<=S2.length-1; j++)
    { S.ch[S1.length+j]=S2.ch[j]; }
  return OK;
}
```

4. 串的基本操作

Status ClearString(HString &S)

将S清为空串

```
{  
    if (S.ch) { free(S.ch); S.ch=NULL; }  
    S.length=0;  
    return OK;  
}
```

4. 串的基本操作

Status SubString(HString &Sub, HString S, int pos, int len)
//用Sub返回串S的第pos个字符开始长度为len的子串

```
{
    if (pos<1 || pos>S.length || len<0 || len>S.length-
        pos+1)
        return ERROR;
    if (!len) { Sub.ch=NULL; Sub.length=0;}
    else {
        Sub.ch=(char *)malloc(len*sizeof(char));
        for (int j=0;j<=len-1;j++) {
            Sub.ch[j]=S.ch[pos-1+j];}
        Sub.length=len;
    }
    return OK;
}
```

4. 串的基本操作

```
Status StrInsert ( HString &S, int pos, HString T )
{ //在串S的第pos个字符之前（包括尾部）插入串T
  if (pos<1||pos>S.length+1) return ERROR; //pos不合法则告警
  if(T.length){ //只要串T不空，就需要重新分配S空间，以便插入T
    if (! (S.ch=(char*)realloc(S.ch, (S.length+T.length)*sizeof(char)) ))
      exit(OVERFLOW); //若开不了新空间，则退出
    for ( i=S.length-1; i>=pos-1; --i ) S.ch[i+T.length] = S.ch[i];
    // 为插入T而腾出pos之后的位置，即从S的pos位置起全部字符均后移
    S.ch[pos-1...pos+T.length-2] = T.ch[0...T.length-1]; //插入T，略/0
    S.length += T.length; //刷新S串长度
  } return OK;
} //StrInsert
```

例1：编写建堆函数（参见教材）

```
Status StrAssign(HString &T, char *chars)
```

```
{ //生成一个串T, T值←串常量chars
```

```
    if (T.ch) free(T.ch); //释放T原有空间
```

```
    for (i=0, c=chars; c; ++i, ++c); //求chars的串长度i
```

```
    if ( !i ) {T.ch = NULL; T.length = 0;}
```

```
        else{
```

```
            if (!(T.ch = (char*)malloc( i *sizeof(char))))
```

```
                exit(OVERFLOW);
```

```
            T.ch[0..i-1] = chars[0..i-1];
```

```
            T.length =i;
```

```
        }
```

```
        Return OK;
```

```
    } //StrAssign
```

c是指针变量，可以自增！意即每次后移一个数据单元。

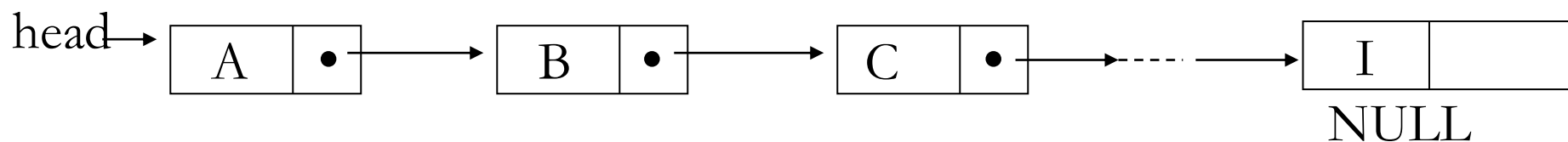
直到终值为“假”停止，串尾特征是
c= '\0'=NULL=0

此处T.ch[0]没有用来装串长，因为另有T.length 分量

3. 串的块链存储表示

- 串的链式存储方式: 用链表存储串值, 易插入和删除。
- 结点大小: 一个或多个字符
 - 存储密度 = 串值所占的存储位 / 实际分配的存储位

法1: 链表结点的数据分量长度取1 (个字符)



法2: 链表结点 (数据域) 大小取n (例如n=4)



讨论: 法1存储密度为 $\frac{1}{2}$; 法2存储密度为 $\frac{9}{15} = \frac{3}{5}$,

显然, 若数据元素很多, 用法2存储更优——称为块链结构

3. 串的块链存储表示

对每个结点的描述

```
#define CHUNKSIZE 80 //每块大小，可由用户定义
typedef struct Chunk { //首先定义结点类型
    char ch[CHUNKSIZE]; //每个结点中的数据域
    struct Chunk * next; //每个结点中的指针域
}Chunk;
```

```
typedef struct { //其次定义用链式存储的串类型
    Chunk *head; //头指针
    Chunk *tail; //尾指针
    int curLen; //结点个数
} Lstring; //串类型只用一次，前面可以不加Lstring
```

对块链表的整体描述

4.3 串的模式匹配算法

- **定义** 在串中寻找子串（第一个字符）在串中的位置
- **词汇** 在模式匹配中，子串称为**模式**，串称为**目标**。
- **示例** 目标 S : “Beijing”
模式 P : “jin”
匹配结果 = 4

典型函数为Index(S,T,pos)

4.3 串的模式匹配算法

字符串模式匹配经典算法：

- BF算法（又称古典的、经典的、朴素的、穷举的）
- KMP算法

主串匹配指针避免回溯，匹配速度快，

主串匹配指针带回溯，速度慢

1. BF算法的实现——即编写Index(S,T,pos)函数

例1： S='ababcabcacbab'， T='abcac'， pos=1，
求：串T在串S中第pos个字符之后的位置。

BF算法设计思想：

- (1) 将主串S的第pos个字符和模式T的第1个字符比较，
若相等，继续逐个比较后续字符；
若不等，从主串S的下一字符 (pos+1) 起，重新与T第一个字符比较。
- (2) 直到主串S的一个连续子串字符序列与模式T相等。返回值为S中与T匹配的子序列第一个字符的序号，即匹配成功。否则，匹配失败，返回值0。

1. BF算法的实现——即编写Index(S,T,pos)函数

```
int Index(SString S, SString T, int pos) {
```

```
    i=pos;    j=1;
```

```
    while ( i<=S[0] && j<=T[0] ) {
```

```
        if (S[i] == T[j] ) {++i, ++j} //继续比较后续字符
```

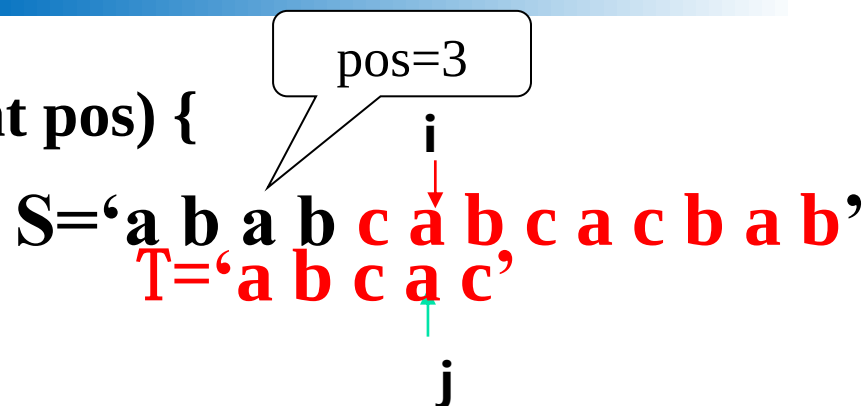
```
        else {i=i-j+2; j=1;} //指针回溯到下一首位，重新开始匹配
```

```
    }
```

```
    if(j>T[0]) return i-T[0]; //子串结束，说明匹配成功
```

```
    else return 0;
```

```
}//Index
```



相当于子串向右滑动一个字符位置

匹配成功后指针仍要回溯！因为要返回的是被匹配的首个字符位置。

算法的时间复杂度计算

最坏情况：若 n 为主串长度， m 为子串长度，则串的匹配算法最坏的情况下需要比较字符的总次数为

$$(n-m+1)*m = O(n*m)$$

最恶劣情况是：主串前面 $n-m$ 个位置都部分匹配到子串的最后一位，即这 $n-m$ 位比较了 m 次，别忘了最后 m 位也各比较了一次，还要加上 m ！所以总次数为： $(n-m)*m + m = (n-m+1)*m$

算法的时间复杂度计算

最好的情况是：一配就中！ 只比较了 m 次。

一般的情况是： $O(n+m)$

推导方法：要从最好到最坏情况统计总的比较次数，然后取平均。

算法的时间复杂度计算

能否利用已部分匹配过的信息而加快模式串的滑动速度？

能！而且主串S的指针i不必回溯！最坏情况也能达到 $O(n+m)$



KMP算法！

2. KMP快速模式匹配

- D. E. Knuth, J. H. Morris, V. R. Pratt同时发现
- 无回溯的模式匹配

KMP算法设计思想

KMP算法的推导过程

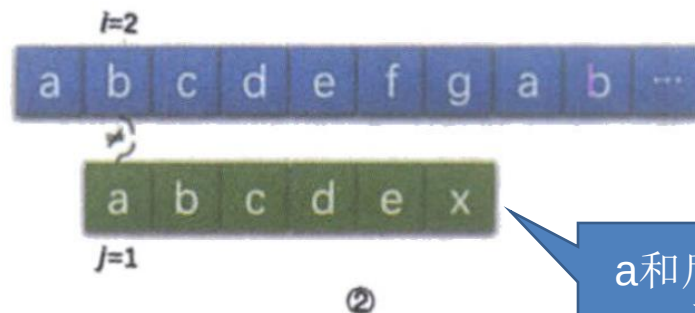
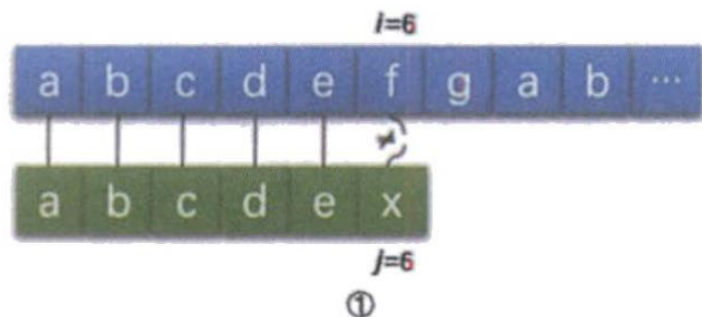
KMP算法的实现（关键技术:计算next[j]）

next值的含义

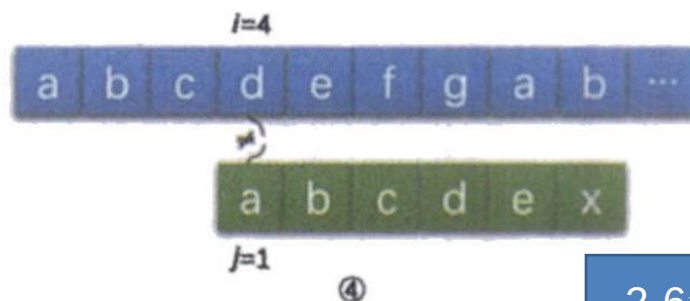
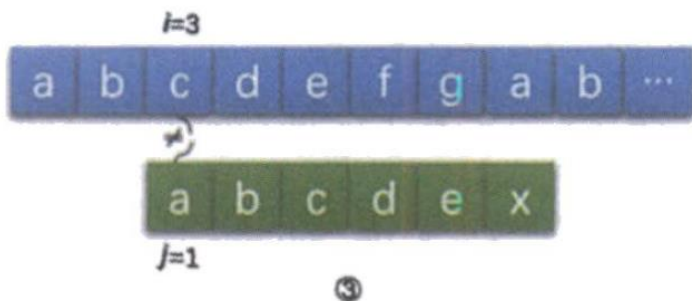
KMP算法的时间复杂度

KMP算法（特点：速度快）

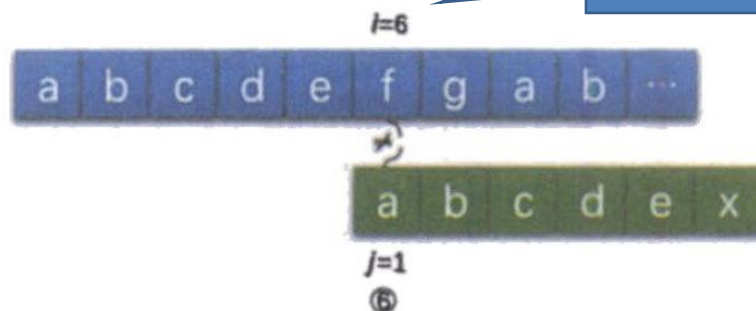
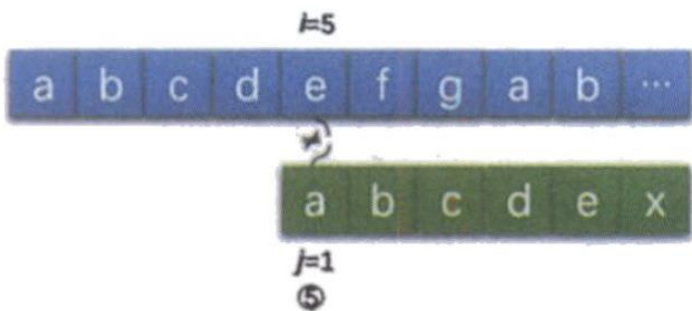
2. KMP快速模式匹配



a和后面串中都不相等

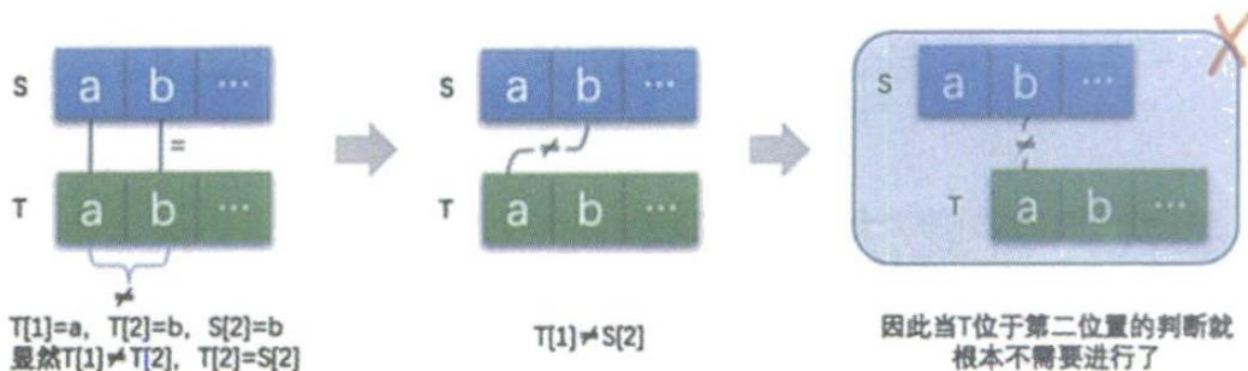


2-6步比较都没意义

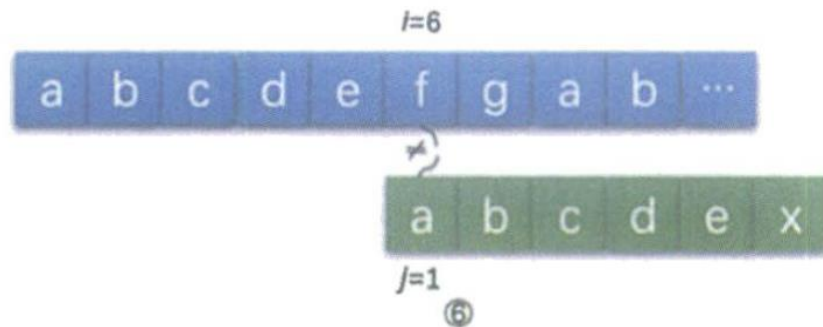
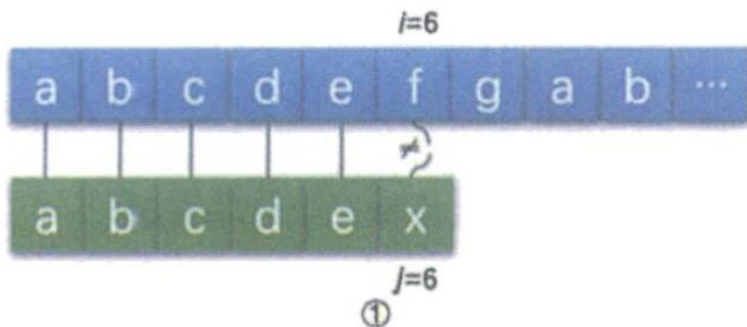


2. KMP快速模式匹配

首字符“a”与T中后面的字符均不相等，且后续字符已与主串进行比较且相等，则可以省略步骤。

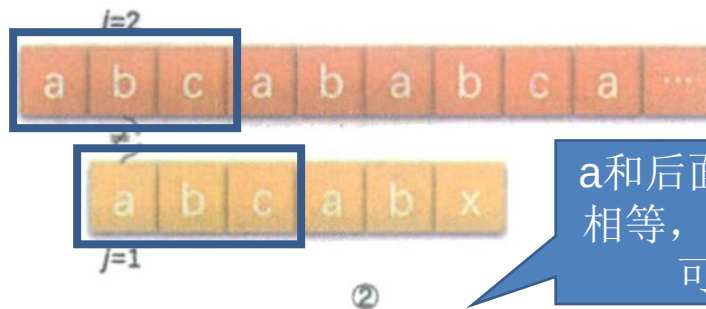
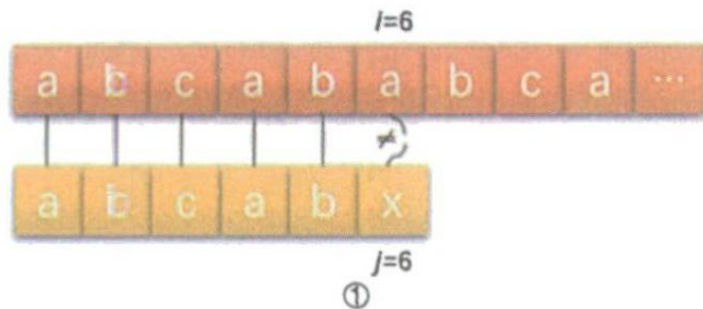


直接从主串中不匹配的位置开始新一轮查找。

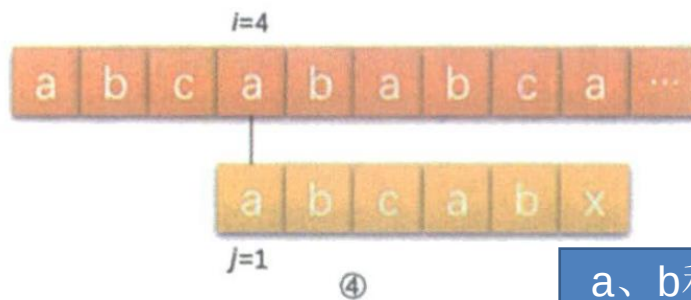
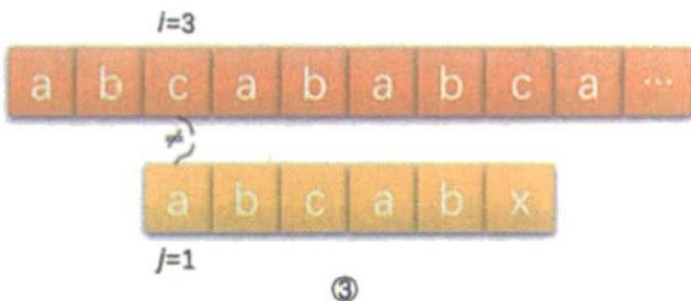


2. KMP快速模式匹配

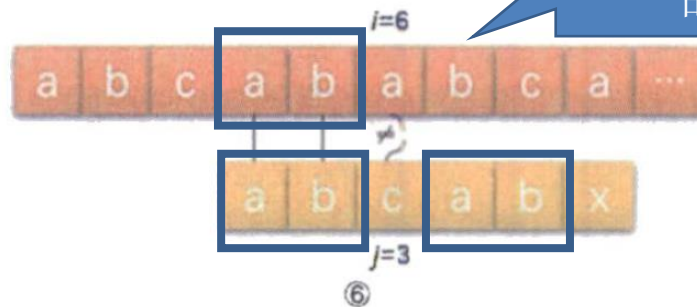
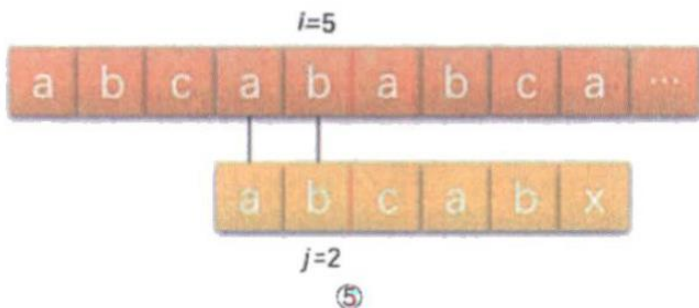
设: $S = \text{"ab cab ab ca"}$ $T = \text{"ab cab x"}$



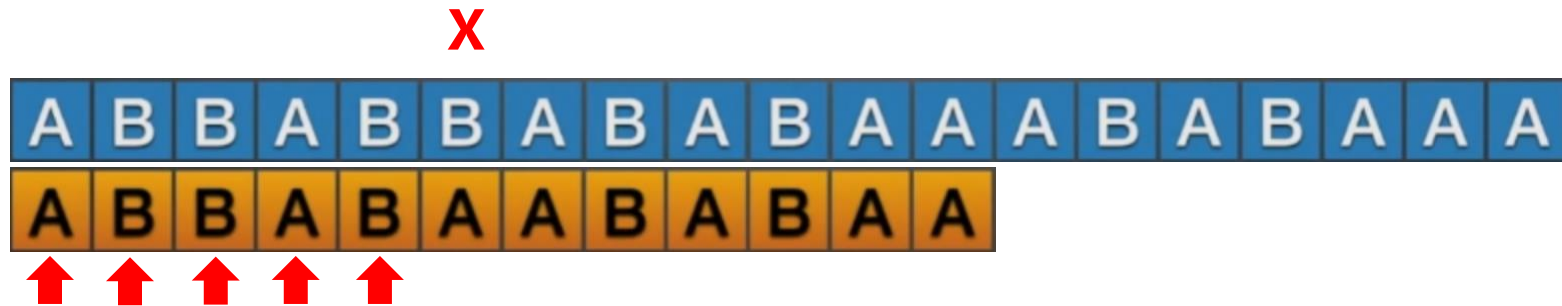
a和后面b、c都不相等，所以2-3步可省略



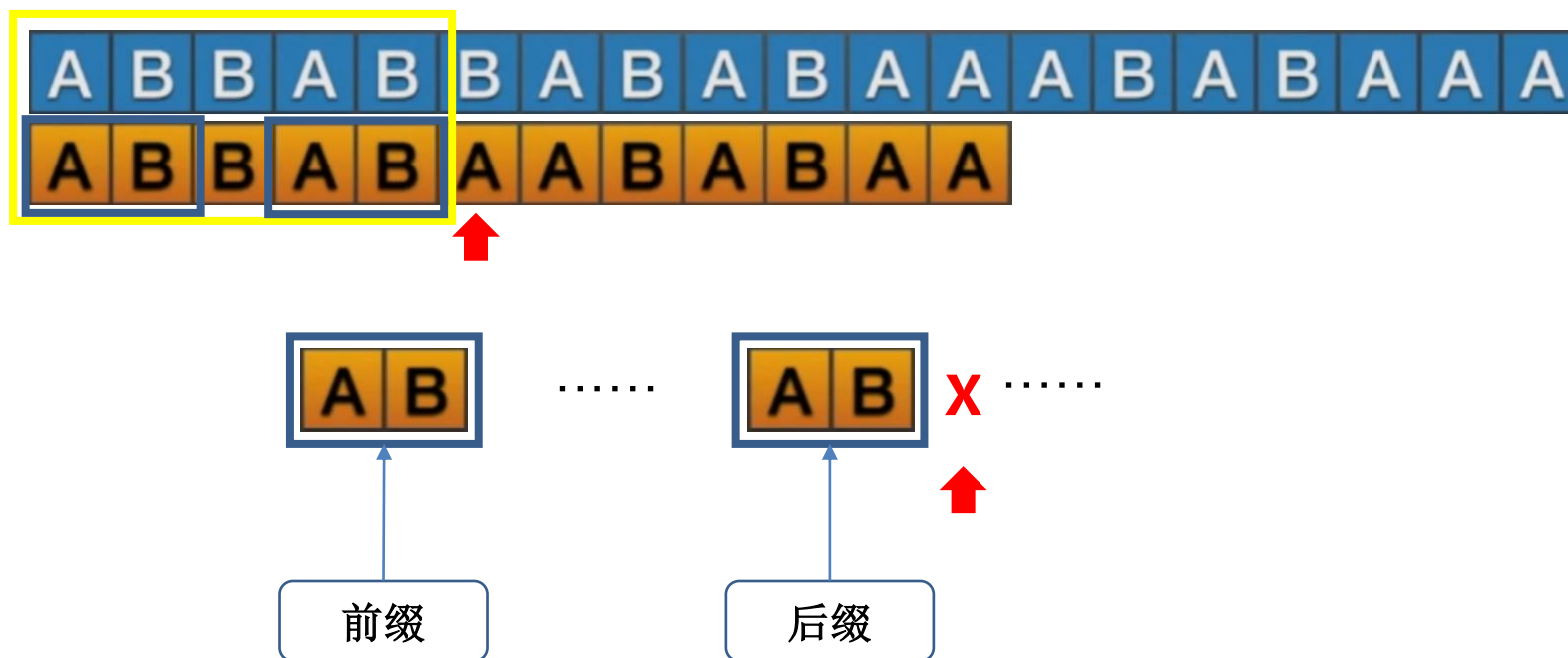
a、b和后面a、b相等，所以5-6步可省略



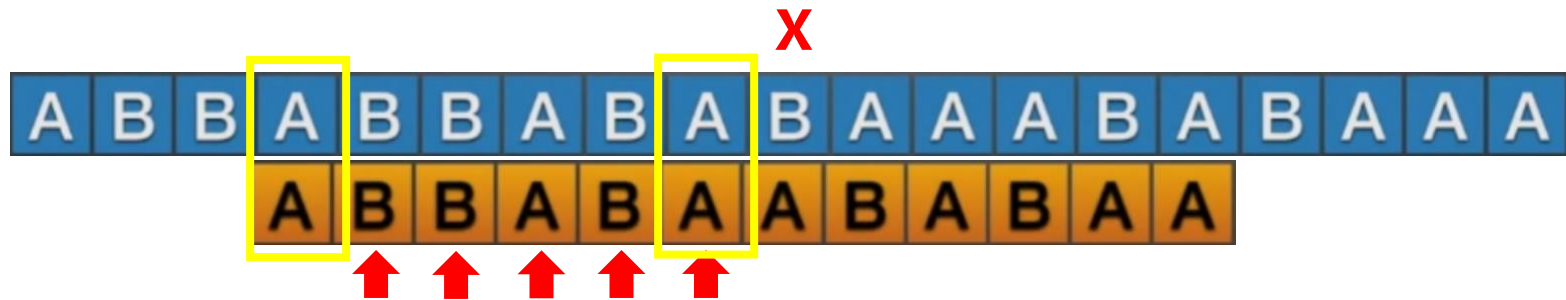
2. KMP快速模式匹配



2. KMP快速模式匹配



2. KMP快速模式匹配

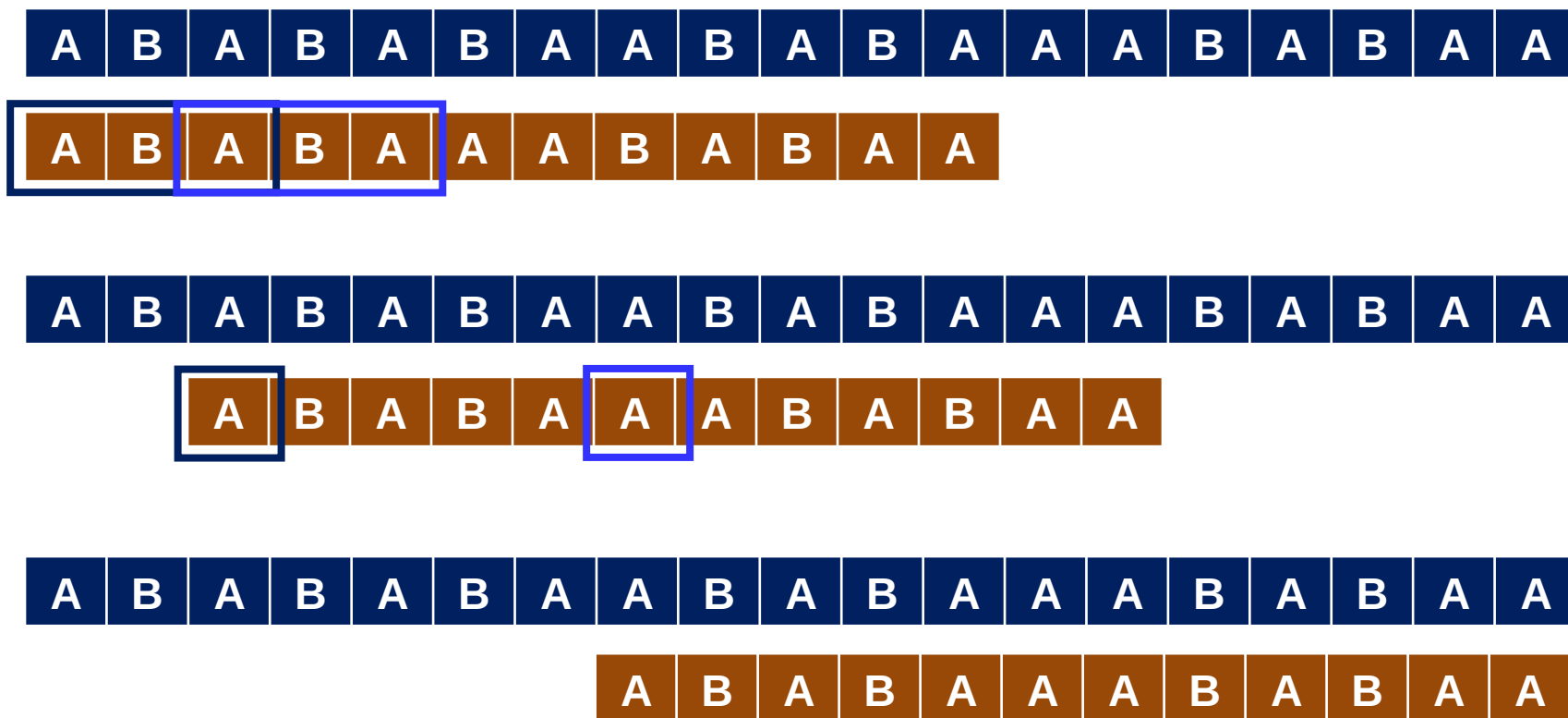


2. KMP快速模式匹配

需要讨论两个问题：

- ① 如何由当前部分匹配结果确定模式向右滑动的新比较起点 k ?
- ② 模式应该向右滑多远才是高效率的?

2. KMP快速模式匹配



2. KMP快速模式匹配

根据模式串T的规律: $'P_1 \dots P_{k-1}' = 'P_{j-(k-1)} \dots P_{j-1}'$

由当前失配位置j(已知), 可以归纳出计算新起点k的表达式。

令 $next[j] = k$ (k 与 j 显然具有函数关系), 则

$$next[j] = \begin{cases} 0 & \text{当 } j=1 \text{ 时} \quad // \text{不比较} \\ \max \{ k \mid 1 < k < j \text{ 且 } 'P_1 \dots P_{k-1}' = 'P_{j-(k-1)} \dots P_{j-1}' \} & \\ 1 & \text{其他情况} \end{cases}$$

取P首之后与Pj处之前
最大的相同子串

2. KMP快速模式匹配

$K = \text{next}[j]$ 物理含义

当主串在*i*处和模式串在*j*处失配时，主串下一次比较位置在失配位置处不变，保持为*i*，主串指针不用回溯；

当主串在*i*处和模式串在*j*处失配时， $\text{next}[j]$ 的含义是用模式串在 $\text{next}[j]$ 所在位置的字符和主串在*i*处的字符重新开始比较， $\text{next}[j]$ 值越小，模式串向右移动越快。

所以kmp算法的关键是如何求模式串所有位置的 next 值。

求next数组的步骤

A	B	A	B	A	A	A	B	A	B	A	A
---	---	---	---	---	---	---	---	---	---	---	---

下标	1	2	3	4	5	6	7	8	9	10	11	12
next	0	1	1	2	3	4	2	2	3	4	5	6

2. KMP快速模式匹配

```
void get_next(SString T, int &next[ ]){ //
```

//求模式串T的next函数值并存入数组next[]。

```
    i=1; next[1]=0; j=0;
```

```
    while(i<T[0] ){
```

```
        if(j==0||T[i]==T[j]){
```

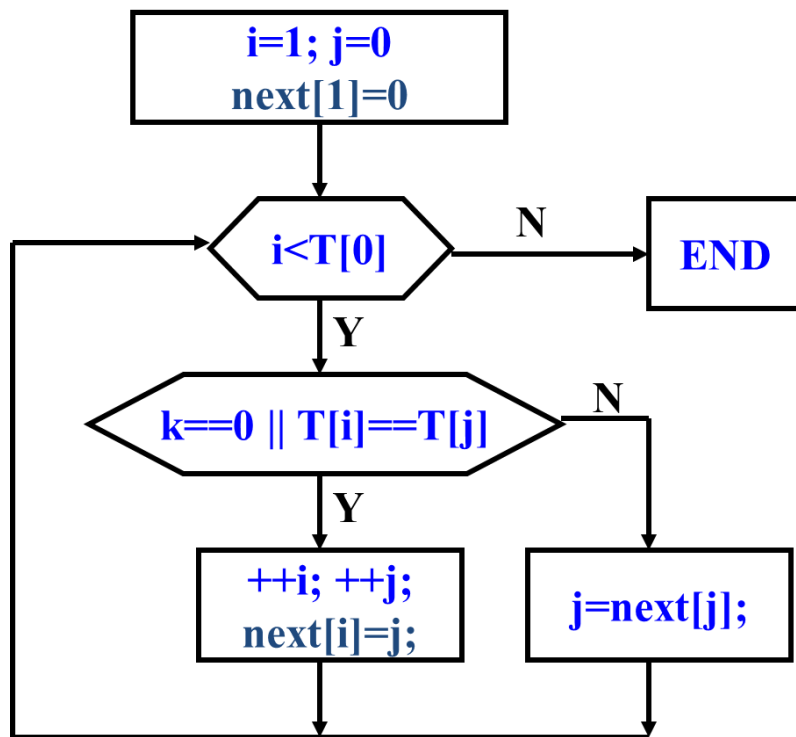
```
            ++i; ++j; next[i]=j;
```

```
        }else
```

```
            j=next[j];
```

```
    }
```

```
}// get_next
```



改进的求next数组方法

next [j]是否完美无缺?

前面定义的next函数在某些情况下还是有缺陷的，
例如模式`aaaab`与主串`aaabaaaab`匹配时的情况：

先计算
next[j]:

j: 1 2 3 4 5
T: a a a a b

next[j]: 0 1 2 3 4

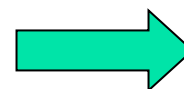
改进的求next数组方法

i: 1 2 3 4 5 6 7 8 9
S: a a a b a a a b
T: a a a a b
 a a a a b b b b

似乎慢了一点?
能否再提速?

此时效率不高的原因为：子串前4位相同时，主串字符若与其中一个不相等，则不必再与其余3个比较。而实际上还在依次比较。

由此派生出next函数的改进算法



改进的求next数组方法

设 $\text{next}[i]=j$

若 $T[i]=T[j]$, 则 $\text{nextval}[i]=\text{nextval}[j]$

```
void get_nextval(SString T, int &nextval[ ]){ //
```

//求模式串T的nextval函数值并存入数组nextval[]。

```
    i=1; nextval[1]=0; j=0;
```

```
    while(i<T[0] ){
```

```
        if(j==0||T[i]==T[j]){
```

```
            ++i; ++j;
```

```
            if(T[i]!=T[j]) nextval[i]=j;
```

```
            else nextval[i] = nextval[j];
```

```
        }else
```

```
            j=nextval[j];
```

```
    }
```

```
}// get_next
```

<i>j</i>	1	2	3	4	5
模式	a	a	a	a	b
<i>next[j]</i>	0	1	2	3	4
<i>nextval[j]</i>	0	0	0	0	4

模式匹配比较

主串S= “00000000002000000000020000000002000000000200000000020000000001”

子串T= “0000000001”

从主串S中查找是否有子串存在，返回对应位置

BF模式匹配算法 循环 285 次 得到查询结果

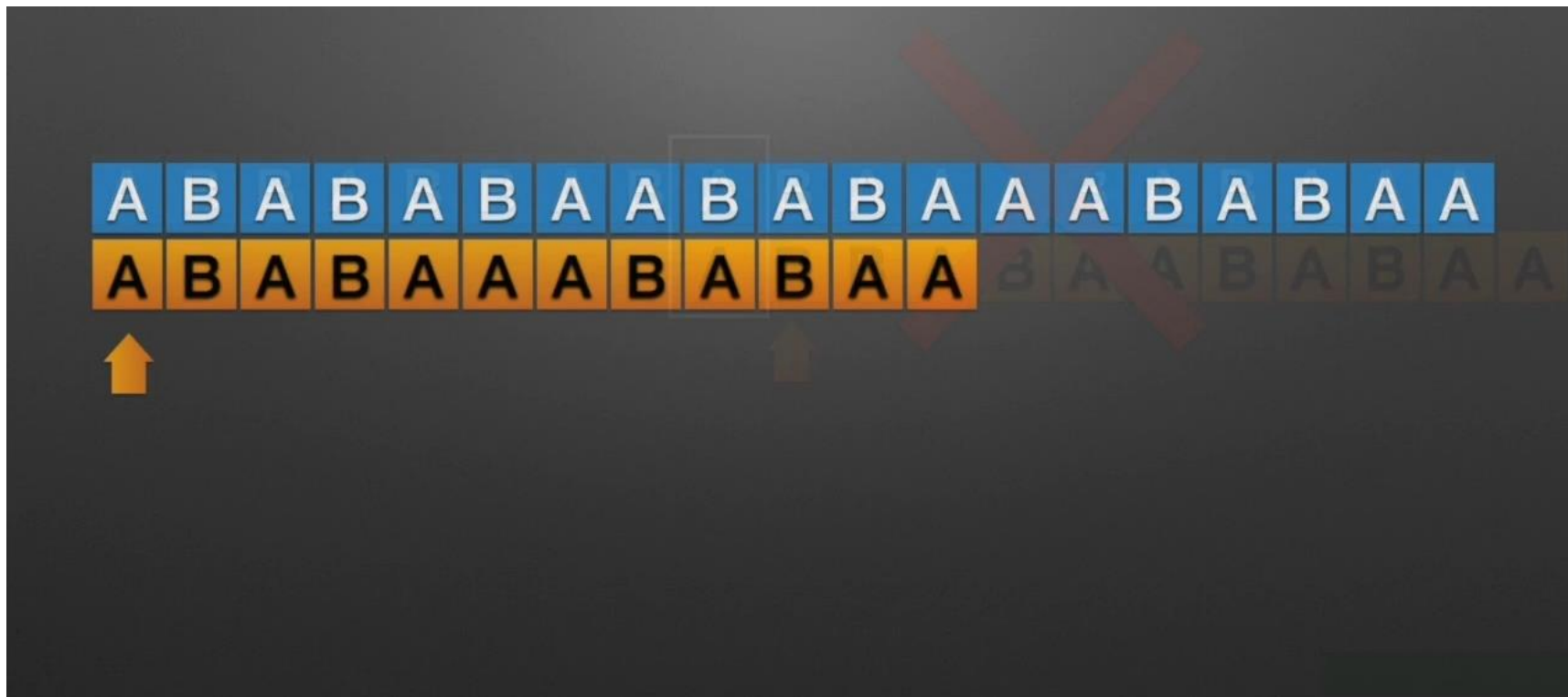
KMP模式匹配算法 循环 110 次 得到查询结果

KMP模式匹配改进算法 循环 70 次 得到查询结果

模式匹配比较

- BF算法的模式匹配算法时间代价：
最坏情况比较 $n-m+1$ 趟，每趟比较 m 次，
总比较次数达 $(n-m+1) * m$
- 原因在于每趟重新比较时，目标串的检测指针要回退。改进的模式匹配算法可使目标串的检测指针每趟不回退。
- 改进的模式匹配(KMP)算法的时间代价：
 - ◆ 若每趟第一个不匹配，比较 $n-m+1$ 趟，总比较次数最坏达 $(n-m) + m = n$
 - ◆ 若每趟第 m 个不匹配，总比较次数最坏亦达到 n
 - ◆ 求next函数的比较次数为 m ，所以总的时间复杂度是 $O(n+m)$

2. KMP快速模式匹配



正在答疑
